Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2024.0429000

How Small Can a Code Model Be? A Reproducible Study of One-Million-Parameter Python Synthesis with Compiler Feedback on Microcontrollers

ROHITH YARRAMALA[1]
[1]Robotics and Automation Group, Defence Institute of Advanced Technology (DIAT), Girinagar, Pune 411025, India (e-mail: yarramala.maecas26@diat.ac.in)
Corresponding author: Rohith Yarramala (e-mail: yarramala.maecas26@diat.ac.in).

Code, data, trained models and the ESP32 firmware: https://github.com/rohithyarramala/microcoder; archived version 1.0.0: https://doi.org/10.5281/zenodo.24187363.

ABSTRACT Can a language model small enough to live in the flash memory of a microcontroller write useful code? We study this question with a fully reproducible setup: a benchmark of IoT-oriented Python function-synthesis tasks with executable unit tests and audited train/test separation, decoder-only Transformers with 0.53 M and 1.05 M parameters, and a portable INT8 inference engine that runs unchanged on a PC, in a browser and on an ESP32-S3. On unseen wordings of tasks from the training families, the 1.05 M model passes the unit tests on 67.9 % of tasks with one-shot greedy decoding (mean of 3 seeds). *Signature anchoring*, which forces the answer to start with the function header copied from the request, raises this to 77.5 %. A compiler-in-the-loop repair procedure (Recursive Error-Feedback, REF) gated by `compile()` and a static undefined-name check reaches 79.8 %. REF mainly fixes syntax: it is no better than resampling under the same budget, and removing the error message from the repair prompt does not hurt. The 0.53 M model is at least as accurate (82.4 % with anchoring, s.d. 1.3) in 588,504 bytes of flash. On task families held out from training, accuracy falls to 1–2 %, and on MBPP to zero. Million-parameter models are therefore practical, verifiable *specialists* for a closed catalogue of embedded utility functions, not general programmers. We release all code, data, models and firmware.

INDEX TERMS TinyML, edge AI, code generation, small language models, program repair, microcontrollers, ESP32, IoT.

I. INTRODUCTION

Language models that write code [1]–[3] have billions of parameters and run in data centres. Many IoT settings cannot rely on that: field devices are often offline, data may not leave the device, and the hardware has megabytes, not gigabytes, of memory [4]–[6]. Even ‘‘small’’ language models such as TinyLlama [7] and MobileLLM [8] have hundreds of millions of parameters. At the other extreme, TinyStories [9] showed that models with a few million parameters can produce fluent text in a narrow domain. This raises a practical question for embedded systems: *what can a code model that fits in microcontroller flash actually do, and which inexpensive techniques make it more reliable?*

Small models make many syntax errors, and a compiler can detect them for free. Feeding compiler diagnostics back to a model is an established idea, from DeepFix [10], DrRepair [11] and Break-It-Fix-It [12] to self-debugging with large models [13]–[15]. Whether it pays off is less clear: Olausson et al. [16] found that, at an equal sampling budget, self-repair is often no better than drawing more samples. We test this question at a scale four to five orders of magnitude smaller than prior work, where every extra generation costs real time on a battery-powered device.

Contributions.

1) **A leak-free embedded code benchmark.** 48 training and 16 held-out task families (sensor scaling, filtering, checksums, bit manipulation, lists, strings and recursion), each with a reference solution and executable tests. A script proves that no test instruction appears in training and that held-out families are never trained on (Section III).
2) **Microcontroller-sized models with an exact budget.** Transformers of 0.53 and 1.05 M parameters (and a 3.05 M configuration) with a 2,048-token tokenizer;

every parameter is accounted for in Table 1. The 0.53 M model needs 588,504 bytes of flash including the tokenizer.

3) **A controlled comparison of cheap reliability techniques** under an equal budget of four generations: signature anchoring, REF with full, line-only or no error message, and resample-until-compiles, plus training-data ablations, two model sizes with three seeds each.
4) **An end-to-end deployment path.** A dependency-free INT8 C engine with an on-device tokenizer, verified against PyTorch; ESP-IDF firmware, a MicroPython module that uses MicroPython's own `compile()` as the verification gate, and an offline "field assistant" application.
5) **Negative results reported as such.** The model does not generalise to unseen task families or to MBPP, and the error message does not help repair.

II. RELATED WORK

Small and on-device language models. TinyML deploys neural networks on microcontrollers through quantisation [17], specialised runtimes [5] and joint architecture–runtime design [6], with standard benchmarks for vision and audio [18]. For language, TinyStories [9] and phi-1 [19] show that narrow, high-quality data lets small models punch above their size; TinyLlama [7] and MobileLLM [8] target phones rather than microcontrollers. We are not aware of code-generation results for models small enough to run from microcontroller flash.

Code generation. Large code models are evaluated by functional correctness on held-out problems such as HumanEval [1] and MBPP [3], often with sample-then-filter decoding [2]. Smaller pretrained encoder–decoders such as CodeT5 [20] still have tens of millions of parameters.

Repair from compiler or execution feedback. DeepFix [10] and DrRepair [11] train models to fix compiler errors; Break-It-Fix-It [12] learns from the model's own mistakes, an idea we test in Section VII-C; Self-Debug [13], Self-Refine [14] and Reflexion [15] prompt large models with feedback. Olausson et al. [16] show that repair must be compared with resampling at equal cost; we follow that protocol. Automated program repair more broadly is surveyed in [21].

III. BENCHMARK

Task families. A family is a parameterised task: four hand-written natural-language phrasings, a canonical reference solution, and a generator of test inputs. A concrete task samples a function name and argument names from small pools and the family's constants (e.g. ADC resolution and reference voltage, filter coefficient, threshold, bit index). Expected outputs are always produced by executing the reference solution. There are 64 families in five categories: *sensor/IoT* (ADC scaling, unit conversion, clamping, range mapping, moving and exponential averages, RMS, median, peak counting, debounce, hysteresis, XOR/additive checksums, byte packing, bit tests), *math*, *list*, *string* and *recursion*. All reference solutions are MicroPython-compatible.

TABLE 1. Model sizes (vocabulary 2,048, context 256 tokens, MLP ratio 4). Every parameter is counted.

	L	d	H	Embedding	Blocks	Total	MiB (INT8)
S	3	96	4	196,608	332,352	529,056	0.50
M	4	128	4	262,144	787,456	1,049,728	1.00
L	6	192	6	393,216	2,656,512	3,049,920	2.91

Splits. With a fixed seed and stratified by category, 16 families (25 %) are held out and never used for training. For the remaining 48 families, phrasing 4 is never used in training. *In-family* test (384 tasks): training families, phrasing 4 only, freshly sampled names and constants. *Out-of-family* test (240 tasks): held-out families, any phrasing. Model selection uses a separate *development* split (Section VII-A). We also report MBPP (500 test problems) [3] as a fully out-of-domain check.

Leakage audit. `scripts/check_leakage.py` verifies: zero exact or normalised instruction overlap between training and both test splits; zero test tasks from a family seen in training for the out-of-family split; zero in-family test instructions sharing any 8-gram with training instructions (4 out-of-family tasks share a generic 8-gram such as "that returns the …"). By construction, 355 of the 384 in-family reference solutions also occur in training: the in-family split tests whether the model recognises a known task from an unseen phrasing, and the out-of-family split tests whether it can write code it has never seen.

Metric. A task passes if the generated program compiles and returns the expected value (floats within 10^{-6}) on all five test inputs, executed in a sandboxed child process with a 2 s CPU limit. We report pass rates with 95 % Wilson intervals, and mean $\pm$ s.d. over three training seeds.

IV. MODEL AND TRAINING

Architecture. A decoder-only Transformer [22] with RMSNorm [23], rotary position embeddings [24], a GELU MLP, no biases, and input/output embeddings tied [25]. The architecture is deliberately plain so that it can be re-implemented in about 300 lines of C. Table 1 gives the exact parameter budget. With a small vocabulary the embedding stays at about a quarter of the model, which is what makes a sub-megabyte INT8 image possible.

Tokenizer. A byte-level BPE [26] with 2,048 entries trained on the training corpus only. Because it is byte-level, any input can be encoded; unseen words simply fall back to short pieces.

Training data. (i) *Generation pairs* (7,664): instruction $\rightarrow$ code. (ii) *Synthetic repair triples* (39,434): instruction, a corrupted program, the exact CPython `SyntaxError` message, and the correct program. Corruptions delete a colon or bracket, break indentation, misspell a keyword, drop an operator or duplicate a token, and are kept only if they really fail to compile. We also tried training-only instruction augmentation (random function/argument names, varied request openings

Algorithm 1 Signature anchoring + REF

Require: instruction I, budget $N = 3$ repairs

$p \leftarrow$ `def name(args):` parsed from I ▷ SA
$C \leftarrow p \,\|\, \text{Greedy}(\langle q\rangle I \langle c\rangle p)$
for $n = 1..N$ **do**
  $E \leftarrow \text{Gate}(C)$ ▷ `compile()`, optionally + name check
  if $E = \emptyset$ **return** C
  $P \leftarrow \langle q\rangle I \langle b\rangle C \langle r\rangle E \langle c\rangle p$
  $C' \leftarrow p \,\|\, \text{Greedy}(P)$; **if** $C' = C$: $C' \leftarrow p \,\|\, \text{Sample}_{0.7}(P)$
  $C \leftarrow C'$
end for
return C if $\text{Gate}(C) = \emptyset$, else Failure

such as "Make a function" or "I need a function"); it lowered accuracy on the development split, so the final recipe does not use it (Section VII-A). The model is trained with the loss on the answer tokens only, on an equal mix of generation and repair examples, with AdamW, a cosine schedule, dropout 0.1, batch 48 and 4,000 steps on two CPU cores (≈40 min).

V. INFERENCE STRATEGIES

All strategies use at most four generations per task, so they are compared at equal worst-case cost.

Signature anchoring (SA). Every request names the function to write. SA extracts the first `name(args)` from the request and forces the answer to begin with `def name(args):`; the model only completes the body. SA adds no parameters and saves the tokens of the header.

Recursive Error-Feedback (REF). Algorithm 1: if the draft does not compile, the model is prompted with the instruction, the faulty draft and the error message, using the same special-token format as the repair training data. If greedy repair returns the draft unchanged, one sample at temperature 0.7 is drawn instead. We also test a second gate, a static undefined-name check described in Section VII-D, applied after `compile()`. To isolate what the message contributes we compare three message variants: the full CPython message; a line-only message ("`SyntaxError: invalid syntax (line n)`"), which is roughly what MicroPython's compiler reports; and no message.

Resample-until-compiles. The equal-budget baseline of [16]: while the program does not compile, draw a fresh sample (temperature 0.7) from the original prompt.

VI. DEPLOYMENT ON AN ESP32-S3

INT8 engine. Weights are quantised per output row to symmetric INT8 with FP32 scales; activations and the KV cache stay FP32 (the ESP32-S3 has a single-precision FPU). The engine is portable C99, performs no allocation in the decode loop and reads weights in place from a constant blob, so on the ESP32-S3 they stay memory-mapped in flash. The tokenizer (GPT-2 pre-tokenisation plus rank-ordered BPE merges) is implemented in the same file. We verified that the C tokenizer reproduces the reference tokenizer on all 18,524 instruction and code strings of the benchmark, the training corpus and MBPP, and that C logits match PyTorch with identically quantised weights to within 10^{-4}.

Memory. Flash holds the model (1,079,840 bytes) and tokenizer (38,840 bytes). RAM holds the runtime state of 1,094,656 bytes, of which 1,048,576 bytes are the FP32 KV cache for the full 256-token context; this requires PSRAM, which most ESP32-S3 modules have. One generated token costs 1,049,600 multiply-accumulates at the first position and 1,310,720 at position 256; the projection onto the 2,048-token vocabulary accounts for 262,144 of these (about one fifth to one quarter), and attention over a full 256-token context for another fifth.

Firmware, simulation and application. The ESP-IDF firmware embeds the model and the benchmark prompts, runs SA + greedy decoding on every prompt and prints the code and timings over UART; a script re-scores that log with the paper's unit tests. The same `app_main()` also runs on a PC under ESP32-S3 memory budgets (host simulation); a Wokwi simulator configuration is also provided, for functional checks only. Built with ESP-IDF v5.4.2 for the ESP32-S3, the firmware image is 870,176 bytes (28 % of its 3,145,728-byte app partition): 682,200 bytes of read-only data in flash, most of it the model and tokenizer, and 119,912 bytes of code; static use of internal RAM is 55,871 of 341,760 bytes (16.3 %). For REF on the device we provide a MicroPython native module; the REF loop is plain MicroPython that uses MicroPython's own `compile()` as the gate. On top of it, a *field assistant* lets a technician describe a sensor function in English over the serial console; the device generates, verifies and installs it, and can immediately stream a live ADC channel through it, all offline.

VII. RESULTS

A. MODEL SELECTION ON THE DEVELOPMENT SPLIT

Our first two attempts were evaluated directly on the test split (Table 4, first two rows). The first used no augmentation; the second replaced the function and argument names by random identifiers in half of the training examples. The second was clearly worse with signature anchoring, so for every later decision we built a separate *development* split: models are trained on phrasings 1–2 only and evaluated on 288 tasks written with phrasing 3 of the training families. Test phrasing 4 is never seen. Table 2 shows that augmentation *hurts*. Varying how a request opens lowers SA+REF accuracy from 79.5 % to 70.8 %, and random identifiers lower it further, to 37.2 % at 50 %. Descriptive names such as `moving_average` are a strong cue to the intended task, and destroying them removes it. The final recipe therefore uses no instruction augmentation. All results below use this recipe and were not tuned on the test split.

B. MAIN RESULTS

Table 3 reports the 1.05 M model over three seeds; the 0.53 M model is compared in Section VII-C.

TABLE 2. Development-split selection (288 tasks, pass %, seed 0, 2,500 steps).

Training data	Greedy	SA	SA+REF
No augmentation	60.8	78.8	79.5
Varied openings	69.8	70.5	70.8
Varied openings + random names (15%)	41.0	43.4	47.6
Varied openings + random names (50%)	26.0	34.0	37.2

Signature anchoring is the largest single gain. Greedy decoding passes 67.9 % of in-family tasks; forcing the header raises this to 77.5 % (seed range 70.8–84.9 %) at zero cost. Without SA the model often writes a correct body under a slightly different name or argument list, which fails every test.

Compiler feedback mostly fixes syntax, not behaviour. REF raises syntactic validity on the in-family split from 95.2 % to 97.7 %, and on the out-of-family split from 69.2 % to 86.0 %. The pass rate barely moves (77.5 % $\rightarrow$ 78.3 %), because most failures are semantic: a wrong constant, an undefined name, a wrong loop bound (Section VII-D).

The error message does not matter, and repair is not better than resampling. With the full CPython message, a line-only message and no message at all, SA+REF reaches 78.3, 78.4 and 78.6 %. Resample-until-compiles reaches 77.9 %. All four differences are far smaller than the seed-to-seed spread (s.d. 6.8 points). This matches the large-model finding of [16] at a scale about 10^4 times smaller. In practice the repair prompt helps because it gives the model a second, conditioned attempt, not because the model reads the diagnostic.

A second, equally cheap gate helps a little. An undefined-name check (Section VII-D) catches failures that `compile()` accepts. Gating REF on `compile()` plus this check gives the best in-family result, 79.8 %, using 1.26 generations on average. The gain over SA alone (77.5 %) is consistent across the three seeds but smaller than the seed-to-seed spread (s.d. 6.0 points).

Out-of-family generalisation is essentially absent. On the 240 tasks from held-out families every strategy passes between 0.7 and 1.2 %. REF still makes most of these programs *compile* (88.2 % valid without SA). The model writes fluent Python, but for tasks it was not trained on, that Python does the wrong thing.

C. ABLATIONS

Repair data also improves generation (Table 4). Removing the repair examples lowers even plain greedy accuracy (45.6 % vs. 59.6 %, seed 0). Repair triples therefore act as useful extra supervision, not only as a skill used at inference time. Adding undefined-name repair examples does not help: 68.8 % with the two-gate REF, against 73.7 % for the final recipe (one seed each, a difference within the seed spread).

Learning from the model's own errors does not apply here. We also tried the Break-It-Fix-It idea [12] of learning from the model's own mistakes, but on training prompts the model produced only 1 syntax error in 15,328 samples at temperature 1.0: its errors appear only on unseen wordings, which we cannot use for training without leaking the test phrasing.

Half a megabyte is enough. Model size matters less than one might expect. Over 3 seeds, the 0.53 M model reaches 82.4 ± 1.3 % with SA and 85.7 ± 2.0 % with SA+REF and both gates, against 77.5 ± 7.1 and 79.8 ± 6.0 % for the 1.05 M model; the 3.05 M model reaches 69.0 % with SA (one seed). For a closed task catalogue, half a megabyte of INT8 weights is enough. Apart from the size comparison, the ablation rows are single runs, and the seed spread of the final recipe (s.d. 7.1 points) is larger than most of these differences, so we do not claim any of them as significant.

Quantisation is lossless in practice. Running the full benchmark through the INT8 C engine instead of FP32 PyTorch gives 70.8 % with SA and 73.4 % with SA+REF and both gates for the 1.05 M model (FP32: 70.8 and 73.7 %), and 81.5 and 83.3 % for the 0.53 M model (FP32: 81.5 and 83.6 %).

D. FAILURE ANALYSIS

For seed 0 with SA+REF on the in-family split, the 108 failures split into 43 runtime exceptions, 23 wrong values, 20 partly correct, and 22 programs that still do not compile. Of the runtime exceptions, 28 are `NameErrors`: the model reuses a variable name from a related family (for instance returning `s` where a constant upper bound was required). This observation motivated the undefined-name gate: a 120-line static check, written without `re` or `ast` so that it also runs in MicroPython, which flags any identifier that is read but never bound. It produces no false positives on all 5,581 reference solutions and flags every `NameError` produced by the model.

E. OUT-OF-DOMAIN: MBPP

On the 500 MBPP test problems the model writes syntactically valid Python for 47.8 % of problems (69.6 % with SA+REF) and passes 0 (greedy) and 0 (SA+REF) of them. This is the expected outcome for a model trained only on 64 templated task families, and it marks the boundary of what such a model should be used for.

F. ON-DEVICE MEASUREMENTS

The firmware path is checked end to end on the host: running the unmodified `app_main()` with the INT8 blob on all 384+240 prompts gives a SA-greedy pass rate of 44.1 %, identical to the FP32 PyTorch result on the same prompts. With 8 MiB of PSRAM the state allocation succeeds; with no PSRAM it fails, as the memory budget predicts. On one x86 core the engine decodes at 0.708 ms per token. Because the 0.53 M model is as accurate as the 1.05 M model (Section VII-C) and needs half the flash and compute, it is the one we deploy. We have not yet measured the model on ESP32-S3 hardware. For orientation only, a simple estimate assuming 4 cycles per INT8 multiply–accumulate at 240 MHz, plus streaming all weights from external flash at 40 MB/s for every

TABLE 3. Pass rate and syntactic validity (%) of the 1.05 M-parameter model, mean ± s.d. over 3 seeds. "Gens" is the mean number of generations used (budget 4).

Strategy	In-family (384 tasks) Valid	In-family (384 tasks) Pass	Out-of-family (240 tasks) Valid	Out-of-family (240 tasks) Pass	Gens
Greedy (one-shot)	94.2±4.2	67.9±12.5	71.4±8.9	0.7±0.2	1.00
Signature anchoring (SA)	95.2±3.7	77.5±7.1	69.2±10.6	1.2±0.5	1.00
REF (no SA)	96.8±3.3	68.6±12.6	88.2±7.1	0.7±0.2	1.13
SA + resample-until-compiles	97.2±3.3	77.9±7.1	81.3±7.6	1.4±0.7	1.12
SA + REF (full message)	97.7±2.9	78.3±6.8	86.0±6.0	1.2±0.5	1.10
SA + REF (line-only message)	97.7±2.8	78.4±6.8	85.7±7.6	1.2±0.5	1.10
SA + REF (no message)	97.7±2.8	78.6±6.7	84.6±7.6	1.4±0.7	1.10
SA + resample, compile+name gate	96.8±3.3	78.5±6.6	73.5±6.4	1.5±0.5	1.29
SA + REF, compile+name gate	97.0±3.3	79.8±6.0	77.2±8.8	1.2±0.5	1.26

TABLE 4. Ablations (pass %, one run each). ID: in-family split; OOD: out-of-family split. SA+REF+name: REF gated by `compile()` and the undefined-name check.

Variant	Params	In-family (ID) Greedy	In-family (ID) SA	In-family (ID) SA+REF	In-family (ID) SA+REF+name	OOD SA+REF+name
First attempt: 500 draws, no dropout	1.05M	3.6	76.3	78.1	79.9	0.0
Second attempt: random names 50%	1.05M	39.3	52.9	53.1	54.4	5.4
No repair data (generation only)	1.05M	45.6	53.9	54.9	54.9	1.2
Final recipe (M, 1.05M params)	1.05M	59.6	70.8	71.9	73.7	1.2
+ undefined-name repair data	1.05M	52.3	65.1	67.2	68.8	1.7
Smaller model (S)	0.53M	72.4	81.5	82.6	83.6	4.6
Larger model (L)	3.05M	50.0	69.0	72.1	72.7	0.0
Final recipe M, INT8 C engine	1.05M	59.6	70.8	71.6	73.4	1.2
Smaller model S, INT8 C engine	0.53M	72.4	81.5	82.6	83.3	4.6

TABLE 5. Deployment. Host values are measured with the same C engine and firmware code; ESP32-S3 values are an analytical estimate (see text) for the 0.53 M model.

	1.05 M (M)	0.53 M (S)
Flash: model + tokenizer (B)	1,118,680	588,504
RAM: state incl. KV cache (B)	1,094,656	626,688
MACs/token, position 1	1,049,600	528,960
MACs/token, position 256	1,310,720	675,840
Host decode (ms/token)	0.708	0.373
Host end-to-end, median (ms)	36.1	19.1
Host SA-greedy pass, all 624 (%)	44.1	51.9
ESP32-S3 decode, *estimate* (ms/token)		10–24
ESP32-S3 on-device accuracy		not measured

token, gives 10–24 ms per token for the 0.53 M model; a board run (`esp32/run_lab.sh board`) replaces this estimate with measured latency and on-device accuracy.

VIII. DISCUSSION AND LIMITATIONS

What a million parameters can do. The model is a reliable *recall-and-adapt* engine. Given an unseen wording of a familiar task, it fills in the right constants, names and structure for roughly four out of five requests, and two free gates make almost all of its output compile. It does not compose new algorithms. This makes it a good fit for a closed catalogue of device-side utilities (unit conversions, filters, thresholds, checksums, register bit operations) that a technician adapts in the field. It is not a replacement for a general coding assistant.

Cheap techniques first. At this scale the decoding procedure matters more than the error feedback: SA gives the largest gain, and the content of the compiler message gives none. For a device with a strict latency budget, SA plus a single gated retry captures nearly all of the available accuracy.

Limitations. (i) The benchmark is synthetic and templated. In-family tests change the wording, names and constants but reuse known solution templates (355 of 384 reference solutions occur in training). (ii) Each family has only four phrasings, and the model is sensitive to wording; real user requests will be more varied than our test phrasing. (iii) Seed-to-seed variance is large (in-family SA pass between 70.8 and 84.9 %), so single-seed ablations are indicative only. (iv) Our first two training attempts were evaluated on the test split before we introduced the development split; we report them rather than hide them. (v) The `compile()` and name gates check form, not function; generated code must still be tested before it controls hardware. (vi) On-device latency is estimated, not measured.

IX. CONCLUSION

We built a fully reproducible pipeline for code generation with Transformers of 0.53 and 1.05 M parameters. The 0.53 M model fits in about half a megabyte of microcontroller flash; on unseen wordings of known embedded tasks it passes the unit tests on 82.4 % of tasks with signature anchoring and 85.7 % with gated repair (1.05 M model: 77.5 and 79.8 %). It does not generalise to new task families. Compiler feedback makes its output compile but adds little correctness, and the

message content is irrelevant. We hope the benchmark, the leakage audit, the INT8 engine and the negative results help others measure on-device code models honestly.

REPRODUCIBILITY

All numbers are generated by `scripts/make_tables.py` from the result files written by the evaluation scripts; `scripts/reproduce.sh` reruns data generation, the leakage audit, training, evaluation, export and the host simulation.

REFERENCES

[1] M. Chen *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.
[2] Y. Li *et al.*, "Competition-level code generation with AlphaCode," *Science*, vol. 378, no. 6624, pp. 1092–1097, 2022.
[3] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, and C. Sutton, "Program synthesis with large language models," *arXiv preprint arXiv:2108.07732*, 2021.
[4] P. Warden and D. Situnayake, *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. O'Reilly Media, 2019.
[5] R. David *et al.*, "TensorFlow Lite Micro: Embedded machine learning for TinyML systems," in *Proc. Machine Learning and Systems (MLSys)*, vol. 3, 2021.
[6] J. Lin, W.-M. Chen, Y. Lin, J. Cohn, C. Gan, and S. Han, "MCUNet: Tiny deep learning on IoT devices," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.
[7] P. Zhang, G. Zeng, T. Wang, and W. Lu, "TinyLlama: An open-source small language model," *arXiv preprint arXiv:2401.02385*, 2024.
[8] Z. Liu *et al.*, "MobileLLM: Optimizing sub-billion parameter language models for on-device use cases," in *Proc. Int. Conf. Machine Learning (ICML)*, 2024.
[9] R. Eldan and Y. Li, "TinyStories: How small can language models be and still speak coherent English?" *arXiv preprint arXiv:2305.07759*, 2023.
[10] R. Gupta, S. Pal, A. Kanade, and S. Shevade, "DeepFix: Fixing common C language errors by deep learning," in *Proc. AAAI Conf. Artificial Intelligence*, 2017, pp. 1345–1351.
[11] M. Yasunaga and P. Liang, "Graph-based, self-supervised program repair from diagnostic feedback," in *Proc. Int. Conf. Machine Learning (ICML)*, 2020.
[12] ——, "Break-it-fix-it: Unsupervised learning for program repair," in *Proc. Int. Conf. Machine Learning (ICML)*, 2021.
[13] X. Chen, M. Lin, N. Schärli, and D. Zhou, "Teaching large language models to self-debug," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024.
[14] A. Madaan *et al.*, "Self-refine: Iterative refinement with self-feedback," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.
[15] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.
[16] T. X. Olausson, J. P. Inala, C. Wang, J. Gao, and A. Solar-Lezama, "Is self-repair a silver bullet for code generation?" in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024.
[17] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 2704–2713.
[18] C. Banbury *et al.*, "MLPerf Tiny benchmark," in *Proc. NeurIPS Datasets and Benchmarks Track*, 2021.
[19] S. Gunasekar *et al.*, "Textbooks are all you need," *arXiv preprint arXiv:2306.11644*, 2023.
[20] Y. Wang, W. Wang, S. Joty, and S. C. H. Hoi, "CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2021, pp. 8696–8708.
[21] C. Le Goues, M. Pradel, and A. Roychoudhury, "Automated program repair," *Communications of the ACM*, vol. 62, no. 12, pp. 56–65, 2019.
[22] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
[23] B. Zhang and R. Sennrich, "Root mean square layer normalization," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.
[24] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu, "RoFormer: Enhanced transformer with rotary position embedding," *Neurocomputing*, vol. 568, p. 127063, 2024.
[25] O. Press and L. Wolf, "Using the output embedding to improve language models," in *Proc. Conf. European Chapter of the Association for Computational Linguistics (EACL)*, 2017, pp. 157–163.
[26] R. Sennrich, B. Haddow, and A. Birch, "Neural machine translation of rare words with subword units," in *Proc. Annual Meeting of the Association for Computational Linguistics (ACL)*, 2016, pp. 1715–1725.

ROHITH YARRAMALA is currently pursuing the M.Tech. degree in robotics and automation at the Defence Institute of Advanced Technology (DIAT), Pune, India. Research interests include tiny machine learning, small language models for embedded systems, on-device code generation, and robotics.